

Plantilla de organización de experimento¹

1. Breve descripción del problema
En el dominio del desarrollo de software de alto rendimiento, la elección de la arquitectura y el compilador ejerce una influencia sustancial en el tiempo de ejecución y el uso de recursos. Sin embargo, la interacción de tales componentes al compilar y ejecutar un conjunto idéntico de funciones en lenguaje C se somete a una evaluación sistemática en raras ocasiones. A través de metodologías de diseño y análisis de experimentos, la presente investigación tiene como propósito cuantificar el impacto de estas variables y su interrelación, proporcionando información de gran valor para la toma de decisiones en proyectos de ingeniería de software.
2. Hipótesis de trabajo (o preguntas de investigación)
¿Cómo afectan la arquitectura de la CPU (Intel vs AMD) y el compilador (GCC vs Clang) al tiempo de ejecución de funciones en C?
3. Objetivos
• Estimar el efecto principal de Arquitectura y de Compilador sobre el tiempo de ejecución (T_{exec}). • Evaluar la interacción Arqu$\times$Comp en T_{exec}. • Cuantificar la variabilidad residual para informar decisiones en entornos de software.

¹ Basado en una plantilla preparada por las profesoras Gabriela Barrantes e Ileana Castillo

4. Variable (s) de respuesta

A. Descriptivo.

Nombre (1)	Descripción (2)	Justificación (3)	Tipo de variable (4)	Unidades (5)
T_exec	Tiempo transcurrido desde el inicio hasta el fin de la ejecución del binario compilado de la función	En esta variable almacenamos el tiempo de ejecución de la batería de pruebas	Continua	Milisegundos

B. Metodológico.

Nombre (1)	Metodología de recolección (2)	Rango de operación (3)	Exactitud y cómo medirla (4)	Precisión y cómo medirla (5)
T_exec	Para cada combinación de Arquitectura (Intel, AMD) × Compilador (GCC, Clang), se ejecuta la misma función 30 veces de forma consecutiva bajo condiciones controladas. Cada ejecución mide el tiempo con <code>clock_gettime(CLOCK_MONOTONIC)</code> restando previamente el overhead obtenido de la calibración. Los 30 tiempos netos se almacenan para el cálculo de medias, sesgo y dispersión.	0 ms – 100,000 ms	Calcularemos el overhead de medición promediando 30 ejecuciones de una función trivial con <code>clock_gettime</code> y restándole de cada tiempo bruto para obtener el neto; definimos el sesgo como la diferencia absoluta entre el tiempo neto medio y ese overhead (se acepta si ≤ 1 ms) y repetimos la calibración al inicio de cada bloque.	Para cada combinación de Arquitectura × Compilador se ejecuta la misma función 30 veces seguidas bajo condiciones idénticas. De esas 30 mediciones se calcula la desviación estándar (σ) y el coeficiente de variación ($CV = \sigma / \text{promedio} \times 100 \%$)

5. Unidad Experimental

Nombre (1)	Descripción (2)	Justificación (3)	Tipo (4)	Unidades (5)
Función C	Función escrita en lenguaje C que implementa una lógica específica; será compilada y ejecutada para medir rendimiento.	Es la unidad elemental de nuestro experimento: al mantener el mismo código en cada prueba, podemos atribuir cualquier cambio en el tiempo de ejecución únicamente a CPU y compilador.	Discreta categórica	N/A

- 1) El nombre de la variable debe ser un mnemónico corto, fácil de entender y específico.
- 2) Dado que el nombre es algo corto, esta es la oportunidad de explicar lo que es.
- 3) La justificación debe hacerse con base en el problema, la hipótesis y los objetivos. Si la justificación es un texto largo, inclúyalo luego de la tabla, con la indicación correspondiente.
- 4) El tipo únicamente puede tener los siguientes valores: continua, discreta numérica o discreta categórica.
- 5) Si la unidad experimental no tiene unidades, indiquen N/A.

6. Factores potenciales de diseño

6.1 Factores de diseño

A. Descriptivo

Nombre	Descripción	Justificación de la inclusión como factor (1)	Justificación como factor de diseño (2)	Tipo y unidades
Arquitectura CPU	Plataforma hardware utilizada para ejecutar el binario	Afecta directamente el rendimiento debido a diferencias en	Fundamental para probar la hipótesis de variabilidad entre plataformas	Discreta categórico (intel, amd)

		sus binarios e manejo de memoria		
Compilador	Herramienta utilizada para generar el binario de la función	Diferentes optimizaciones afectan el desempeño	Necesario para medir la influencia del compilador en el rendimiento	Discreta categórica (GCC, Clang)

(1) Debe explicar por qué usted considera que este factor tiene alguna influencia en la variable de respuesta escogida

(2) Debe explicar por qué decidió utilizarlo como factor de diseño, con base al planteamiento del problema, la hipótesis de trabajo y los objetivos.

B. Metodológico.

Nombre	Metodología de aplicación (1)	Rango de Operación	Niveles a usar	Justificación de los niveles a usar (2)	Medida de precisión y exactitud al fijar los niveles (3)
Arquitectura a CPU	Uso de hardware real o emuladores para cada arquitectura	Intel i5-1235U, AMD Ryzen 5 5500U (Generaciones y familias equivalentes)	2	Representan arquitecturas populares y relevantes actualmente	- Precisión: Cada vez que “cambiamos” de arquitectura las condiciones de inicio y el estado del hardware sean lo más reproducibles posible. Definimos un mini-benchmark de arranque. Si repetimos ese mini-benchmark 10 veces en la misma máquina recién arrancada, la desviación estándar de esos tiempos debe

					ser ≤ 0.5 s. Si cumple, consideramos que hemos “fijado” la arquitectura con buena precisión. - Exactitud: Antes de cada bloque de pruebas ejecutamos <code>lscpu</code> (o <code>dmidecode</code>) y comprobamos que el “Model name” y la “CPU MHz” coincidan con la ficha técnica de fábrica (p. ej. ± 50 MHz de la frecuencia nominal).
Compilador	Selección del compilador instalado apropiadamente para cada arquitectura	GCC, Clang	2	Amplia representación de ecosistemas de compiladores C	- Exactitud en flags: validar con <code>gcc --version</code> y <code>clang --version</code>. - Precisión: repetir compilación 5 veces y medir desviación del binario resultante (hash idéntico y tamaño ± 0 bytes).

- (1) Debe explicar, metodológicamente, cómo se aplica el factor. Dado que puede ser un texto relativamente largo, puede extraerlo de la tabla.
- (2) Los niveles a usar básicamente definen los tratamientos a aplicar, por lo que debe justificarlos con respecto al planteamiento del problema, la hipótesis y los objetivos.

- (3) Precisión y exactitud al aplicar los niveles. Esto es, si decide aplicar X, dada la metodología de aplicación, qué tan precisa y qué tan exacta es esta aplicación.

6.2 Factores que se mantendrán constantes

A. Descriptivo

Nombre	Descripción	Justificación de la inclusión del factor (1)	Justificación como factor constante	Tipo y unidades
Código fuente	Funciones escritas en C	Garantizar comparabilidad	Para atribuir diferencias únicamente a arquitectura o compilador	no aplica

(1) Debe explicar por qué usted considera que este factor tiene alguna influencia en la variable de respuesta escogida

B. Metodológico

Nombre	Metodología de fijación(1)	Rango de Operación (2)	Nivel de fijación	Justificación del nivel fijado (3)	Medida de precisión y exactitud al fijar el nivel
Sistema operativo	Se usará la misma versión del sistema operativo entre ejecuciones de pruebas	ubuntu 24.02	1	no es parte del experimento probar diferentes sistemas operativos	Chequeo previo con <code>lsb_release</code> cada día de prueba.
Versión del compilador e flags de compilación	usar el misma versión del compilador en ambas arquitecturas, y usar mismo commando	gcc 15.1	1	Para aislar efecto del compilador puro sin variación de flags.	Script automatizado que imprime versión y flags antes de cada corrida.
Cantidad de programas abiertos en primer plano	Se limitará a mantener solamente la ejecución de las pruebas en primer plano	1	1	Lo ideal es tener la misma carga de trabajo en todas las pruebas	Medir uso de CPU y RAM antes de cada bloque; debe ser $\leq 5\%$

- (1) Cómo fija el factor y se asegura que se mantenga en el valor decidido.
- (2) Posibles valores
- (3) Justificación de que tenga sentido que este nivel se mantenga fijo de acuerdo con el problema, la hipótesis y los objetivos.

6.3 Factores a los que se permitirá variar

A. Descriptivo

Nombre	Descripción	Justificación de la inclusión del factor (1)	Justificación como factor al que se permitirá variar	Tipo y unidades
Carga del sistema	Procesos en segundo plano y uso de recursos del sistema durante la ejecución	La carga del sistema puede afectar los tiempos de ejecución al competir por recursos de CPU y memoria	No se controlará específicamente para reflejar condiciones más realistas de uso en entornos de producción donde la tarea no se escucha	Continua (% de uso de CPU/memoria)
Temperatura de CPU	Temperatura del procesador durante la ejecución	La temperatura puede influir en el rendimiento por los mecanismos de throttling térmico	Se permite variar naturalmente para representar condiciones reales de operación	Continua (°C)
Tamaño de datos	Volumen de datos procesados por la función	El tamaño de los datos puede afectar los patrones de acceso a memoria y caché	Se medirá su efecto pero no se controlará estrictamente	Discreta numérica (bytes)

(1) Debe explicar por qué usted considera que este factor tiene alguna influencia en la variable de respuesta escogida

7. Factores de molestia

Nombre	Descripción	Justificación de la inclusión del factor	Justificación como factor al que se permitirá variar	Tipo y unidades
Código con librerías dinámicas	Bibliotecas compartidas cargadas en tiempo de ejecución.	Pueden variar la latencia de enlace dinámico.	Se minimiza con preload fijo y caché de bibliotecas.	Catórica (sí/no)
Interrupciones de CPU y cambios de contexto	Eventos de sistema que detienen el proceso y redistribuyen tiempo CPU.	Alteran mediciones de tiempo de ejecución.	Se registran con <code>perf record</code> y se aleatoriza el orden de pruebas.	Continúa (eventos/minuto)
Materiales de construcción y enfriamiento de las máquinas (Posible estrangulamiento térmico)	Temperatura del procesador durante la ejecución	La temperatura puede influir en el rendimiento por los mecanismos de throttling térmico	Se permite variar naturalmente para representar condiciones reales de operación	Continua (°C)

- (1) Los tipos de molestia, según la clasificación de Montgomery son: controlables y no controlables, y dentro de los no controlables, medibles y no medibles.
- (2) Las formas estándar de minimización son aleatorización, replicación y bloqueo. Además -si corresponde- pueden incluir estrategias específicas, por ejemplo, llevar a cabo los experimentos en locales sin contaminación, apagar todos los procesos antes de medir tiempos en un computador, y otros.

8. Restricciones

- Solo se disponen de dos máquinas físicas (Intel y AMD).
- No se probarán sistemas operativos ni flags adicionales.

- Máximo de 5 ejecuciones por combinación debido a tiempo computacional.
- No se incluyen arquitecturas virtuales/emuladas.

9. Aspectos de diseño (aleatorización, bloqueo, ...)

- Diseño factorial 2^2 (2 arquitecturas $\times$ 2 compiladores).
- Dos réplicas completas (10 ejecuciones por combinación).
- Aleatorización: orden aleatorio de las corridas.
- Bloqueo: ejecutar ambas réplicas por día dentro de cada máquina

10. Corridas de prueba

- Se realizaron pruebas preliminares con 3 funciones de prueba para verificar scripts de automatización y medición.
- Ajustes: confirmación de sincronización de relojes y limpieza de caché entre ejecuciones.
- No hay datos de interés adicional para el experimento principal.